
Commande de l'UR5 avec ROS2

Professeur encadrant : Sylvain Durant

BINDER Aurore
HATTON Axel
SCHMITT Théo

Mécatronique 5 - Semestre 9

Table des matières

1	Installation	2
2	Installing d'URCap sur le robot	2
3	Connexion au robot	2
4	Bouger le robot en envoyant des consignes de position articulaires	2
5	Contrôle des positions articulaires du robot	3
5.1	Création d'un package	3
5.2	Test du package sur le robot et en simulation	4
6	Cinématique inverse du robot avec MoveIt	5

Introduction

1 Installation

Tuto détaillé sur le site d'UR : https://docs.ros.org/en/ros2_packages/humble/api/ur_robot_driver/doc/installation/installation.html et https://docs.universal-robots.com/Universal_Robots_ROS2_Documentation/doc/ur_robot_driver/ur_robot_driver/doc/installation/installation.html

Après avoir installé ROS2, il faut installer le driver UR. Pour cela :

```
sudo apt-get install ros-${ROS_DISTRO}-ur
```

On peut aussi installer le package suivant pour ROS control :

```
sudo apt-get install ros-${ROS_DISTRO}-ros2controlcli
```

Lien Github : https://github.com/UniversalRobots/Universal_Robots_ROS2_Driver/?tab=readme-ov-file

2 Installing d'URCap sur le robot

Tuto détaillé d'UR : https://docs.ros.org/en/ros2_packages/humble/api/ur_robot_driver/doc/installation/install_urcap_cb3.html#install-urcap-cb3

3 Connexion au robot

Connecter le robot à l'ordinateur par câble ethernet. Désactiver le wifi sur l'ordinateur.

Sur le robot, aller dans Setup Robot puis Network et noté l'adresse IP du robot ainsi que le masque du sous-reseau.

Sur l'ordinateur, allé dans l'application Settings puis dans Network. Dans Wired, cliquez sur l'icone avec l'engrenage, puis allez dans IPv4. Choisissez Manual pour IPv4 Method, et entré pour le PC une adresse IP proche de celle du robot, et le même Netmask.

Éteignez Ethernet sur le robot.

On peut maintenant essayer de se connecter au robot. Pour cela, entrer la commande suivante :

```
ros2 launch ur_robot_driver ur_control.launch.py ur_type:=ur5 robot_ip:=<IP_OF_THE_ROBOT>  
launch_rviz:=false
```

Une fois que plus rien ne bouge, lancer le programme sur l'ur5 (n'oubliez pas d'éteindre ethernet sur le robot!!).

4 Bouger le robot en envoyant des consignes de position articulaires

Le robot est maintenant près à bouger. Pour un test rapide, utilisez :

```
ros2 launch ur_robot_driver test_scaled_joint_trajectory_controller.launch.py
```

Vous devrez d'abord mettre le robot dans les positions articulaires suivantes :

```
shoulder_pan_joint: 0  
shoulder_lift_joint: -1.57  
elbow_joint: 0  
wrist_1_joint: -1.57  
wrist_2_joint: 0  
wrist_3_joint: 0
```

Ces positions de départs n'étant pas très pratique, on peut créer notre propre fichier launch avec la position de départs et la trajectoire que nous voulons.

5 Contrôle des positions articulaires du robot

5.1 Création d'un package

https://insaros.github.io/ROS2/tuto_2_packages_python_cpp.html#section4

Nous allons commencer par créer notre workspace en tapant les commandes suivantes :

```
mkdir ~/ros2_ws/src/  
cd ~/ros2_ws/  
colcon build
```

Nous allons ensuite créer notre propre package :

```
cd ~/ros2_ws/src/  
ros2 pkg create --build-type ament_python <package_name>
```

Notre but est d'envoyer au robot des positions articulaires, et nous allons nous inspirer du fichier `test_scaled_joint_trajectory_controller.launch.py` fournis par ur. Voici le lien vers le github : https://github.com/UniversalRobots/Universal_Robots_ROS2_Driver/blob/main/ur_robot_driver/launch/test_scaled_joint_trajectory_controller.launch.py

En lisant le code, on constate que ce fichier appelle un noeud de `ros2` `controll`, qui provient du package `ros2_controllers_test_nodes`. Ce noeud prend en paramètres les positions articulaires de la trajectoire, qui sont lues dans un fichier `test_goal_publishers_config.yaml`.

Nous allons commencer par créer le fichier `yaml`. Dans `ros2_ws/src/notre_package` nous créons un dossier `config`, dans lequel on crée notre fichier `.yaml`. On peut utiliser le modèle suivant,

```
publisher_scaled_joint_trajectory_controller:  
  ros__parameters:  
  
    controller_name: "scaled_joint_trajectory_controller"  
    wait_sec_between_publish: 6  
  
    goal_names: ["pos1", "pos2", "pos3", "pos4"]  
    pos1:  
      positions: [0.0, -1.84, -1.95, -0.92, 1.57, 0.0]  
    pos2:  
      positions: [0.0, -2.09, -1.57, 0.42, 1.67, 0.0]  
    pos3:  
      positions: [0.0, -2.2, -1.57, 0.64, 1.67, 0.0]  
    pos4:  
      positions: [0.0, -1.57, -1.57, -1.57, 1.57, 0.0]  
  
    joints:  
      - shoulder_pan_joint  
      - shoulder_lift_joint  
      - elbow_joint  
      - wrist_1_joint  
      - wrist_2_joint  
      - wrist_3_joint  
  
    starting_point_limits:  
      shoulder_pan_joint: [-0.1,0.1]  
      shoulder_lift_joint: [-1.6,-1.5]  
      elbow_joint: [-1.6,-1.5]  
      wrist_1_joint: [-1.6,-1.5]  
      wrist_2_joint: [1.5,1.6]  
      wrist_3_joint: [-0.1,0.1]
```

Il faut juste changer les positions et les limites des points de départs conformément à ce qu'on veut.

Créons maintenant notre fichier launch. Dans `ros2_ws/src/notre_package` on crée un dossier `launch`, et on crée un fichier qui finit par `.launch.py`. Nous allons remplir ce fichier en copiant le code du fichier `test_scaled_joint_trajectory_controller.launch.py` fournis par ur (lien vers le github : https://github.com/UniversalRobots/Universal_Robots_ROS2_Driver/blob/main/ur_robot_driver/launch/test_scaled_joint_trajectory_controller.launch.py)

Il faut évidemment changer le nom du fichier `.yaml` conformément à celui que nous avons créé (ligne 46 du code).

La dernière étape consiste à modifier le fichier `setup.py` qui se trouve dans `ros2_ws/src/notre_package`. Ouvrez ce fichier et ajoutez dans `data_files = [...]` les deux lignes suivantes :

```
('share/' + package_name + '/launch', ['launch/nom_du_fichier_launch.launch.py']),
('share/' + package_name + '/config', ['config/nom_du_fichier_yaml.yaml']),
```

On peut maintenant essayer de build notre nouveau package ! Placez vous dans `~/ros2_ws` et faites la commande `colcon build`. Sourcez ensuite l'overlay avec la commande `source install/setup.bash`

5.2 Test du package sur le robot et en simulation

Pour tester votre package sur le robot, on procède comme précédemment. Ouvrez un nouveau terminal et entrez la commande suivante :

```
ros2 launch ur_robot_driver ur_control.launch.py ur_type:=ur5 robot_ip:=<IP_OF_THE_ROBOT>
launch_rviz:=false
```

Puis dans le terminal où vous avez sourceé l'overlay, lancez la commande suivante :

```
ros2 launch nom_du_package nom_du_fichier_launch.launch.py
```

Il est également possible de d'abord tester votre package en simulation, si vous n'avez pas le robot à votre disposition. Pour cela, lancez le robot grâce à la commande suivante :

```
ros2 launch ur_robot_driver ur_control.launch.py ur_type:=ur5 robot_ip:=yyy.yyy.yyy.yyy
use_fake_hardware:=true launch_rviz:=true
```

puis lancez le fichier launch comme si c'était pour un vrai robot.

Vous pouvez toutefois ajouter un argument :

```
ros2 launch nom_du_package nom_du_fichier_launch.launch.py check_starting_point:=false
```

Le robot ne vérifiera pas si il est dans la configuration de base fournie dans le fichier `yaml` ou pas, ce qui n'est pas forcément nécessaire en simulation.

Si vous voulez que le robot en simulation démarre toujours dans une certaine position, vous devez modifier le fichier suivant : `/opt/ros/humble/share/ur_description/config/initial_positions.yaml`

Placez vous dans le bon répertoire et modifiez le avec `sudo nano`. La simulation démarrera alors toujours avec cette configuration.

Le nœud que nous appelons dans le fichier launch va d'abord vérifier l'état des joints, pour vérifier que le robot n'est pas trop éloigné de sa position initiale définie dans le fichier `yaml`. Pour cela, il lit l'état des joints sur le topic `/joint_states`.

Il va ensuite parcourir les différents points articulaires à atteindre fournis dans le fichier `yaml`, et les envoyer une à une au topic `joint_trajectory`.

Voici lien vers le nœud appelé : https://github.com/ros-controls/ros2_controllers/blob/master/ros2_controllers_test_nodes/ros2_controllers_test_nodes/publisher_joint_trajectory_controller.py

6 Cinématique inverse du robot avec MoveIt

https://moveit.picknik.ai/main/doc/tutorials/getting_started/getting_started.html

```
ros2 launch ur_moveit_config ur_moveit.launch.py ur_type:=ur5e launch_rviz:=true
```

```
ros2 run moveit_test moveit_essaie
```